

KMP算法的理解: A. 如何匹配

①

T	0	1	2	3	4	5	6	7	8	9	10
A	B	A	A	B	A	B	A	B	C	A	
P	A	B	A	B	C						
Next	0	0	1	2	0						

断点前next为1

①.5

T	0	1	2	3	4	5	6	7	8	9	10
A	B	A	A	B	A	B	A	B	C	A	
P	A	B	A	B	C						
Next	0	0	1	2	0						

↑
j的起始点就是刚才的next, 即1

②

T	0	1	2	3	4	5	6	7	8	9	10
A	B	A	A	B	A	B	A	B	C	A	
P	A	B	A	B	C						
Next	0	0	1	2	0						

断点前next为0, 即下文j=0

③

T	0	1	2	3	4	5	6	7	8	9	10
A	B	A	A	B	A	B	A	B	C	A	
P	A	B	A	B	C						
Next	0	0	1	2	0						

下轮i=7
下轮j=2

④

T	0	1	2	3	4	5	6	7	8	9	10
A	B	A	A	B	A	B	A	B	C	A	
P	A	B	A	B	C						
Next	0	0	1	2	0						

匹配成功

B. 如何计算next数组: 它和以前相同的真前缀长度

eg ①

0	1	2	3	4
A	B	A	B	A
0	0			

②

0	1	2	3	4
A	B	A	B	A
0	0			

③

0	1	2	3	4
A	B	A	B	A
0	0	1		

④

0	1	2	3	4
A	B	A	B	A
0	0	1	2	

⑤

0	1	2	3	4
A	B	A	B	A
0	0	1	2	3

这是思路, 那算法呢?

①

0	1	2	3	4
A	B	A	B	A
0	0			

Pass (2, 1, 0)

②

0	1	2	3	4
A	B	A	B	A
0	0	1		

$P[2]=P[0], k++ (0 \rightarrow 1), next[2]=k=1$

③

0	1	2	3	4
A	B	A	B	A
0	0	1	2	

在k=1的基础上, $P[3]=P[1]=P[k], k++ (1 \rightarrow 2)$
 $next[3]=k=2$

④

0	1	2	3	4
A	B	A	B	A
0	0	1	2	3

$P[4]=A=P[2], k++ (2 \rightarrow 3), next[4]=k=3$

3. next 数组的计算过程 (动态规划思想)

计算 next 数组的过程本身就像一个模式匹配过程, 是模式串自己匹配自己:

我们使用两个指针:

- i : 正在计算的后缀的末尾索引 (即当前要求 $next[i]$).
- k : 已经匹配的前缀的长度 (即当前前缀的 $next$ 值, 也就是下一个可能匹配的位置).

初始化:

- 模式串长度为 m .
- $i = 1$ (从第二个字符开始计算).
- $k = 0$ (长度为1的前缀 $P[0]$ 的 $next$ 值为0).
- $next[0] = 0$.

迭代过程 (以 $P[i]$ 为例):

- 匹配成功:
 - 如果 $P[i] = P[k]$, 说明我们找到了一个更长的相等前后缀.
 - k 增加1.
 - $next[i] = k$.
 - i 增加1, 继续考察下一个位置.
- 匹配失败:
 - 如果 $P[i] \neq P[k]$, 说明当前的前缀 $P[0..k-1]$ 和后缀 $P[i-k..i-1]$ 无法扩展.
 - 这时, 我们需要利用已有的 $next$ 信息进行回溯: 令 $k = next[k-1]$.
 - 回溯后, 我们继续比较 $P[i]$ 和新的 $P[k]$, 直到 $k = 0$ (回到了开头) 或匹配成功为止.